

MCTS with a Policy-Value Network

Thuong Dang

May 2026

High-Level Idea

Goal:

Train an agent to play generalized Tic-Tac-Toe by self-play.

The agent combines two components:

Neural network intuition + MCTS search

The neural network answers:

What moves look promising?

How good is this position?

MCTS uses these answers to search more effectively.

Core Learning Loop

The system improves itself through repeated self-play:

Network $\rightarrow$ MCTS $\rightarrow$ Self-play data $\rightarrow$ Training $\rightarrow$ Better network

More explicitly:

$f_{\theta}(s) \rightarrow$ raw policy/value

$\text{MCTS}_{\theta}(s) \rightarrow$ stronger search policy

$(x, \pi, z) \rightarrow$ training examples

The key idea:

Search improves decisions; training distills search into the network.

Game State

A game state is

$$s = (B, c)$$

where

$$B \in \{-1, 0, +1\}^{N \times N}$$

is the board, and

$$c \in \{-1, +1\}$$

is the current player.

Legal actions:

$$A(s) = \{a : B[a] = 0\}$$

Transition:

$$T(s, a) = s' = (B', -c)$$

where action a places player c 's mark on the board.

Observation Encoding

The network sees the board from the current player's perspective:

$$x = \text{encode}(B, c)$$

$$x = [x_{\text{my}}, x_{\text{opp}}]$$

where

$$x_{\text{my}}[i] = \mathbf{1}\{B[i] = c\}$$

$$x_{\text{opp}}[i] = \mathbf{1}\{B[i] = -c\}$$

This encourages symmetric learning:

“my pieces” vs. “opponent pieces”

not

“X pieces” vs. “O pieces”.

Policy-Value Network

The neural network is a function

$$f_{\theta}(x) = (\ell, v)$$

where

$$\ell \in \mathbb{R}^{N^2}$$

are policy logits, and

$$v \in [-1, 1]$$

is the value estimate.

The policy distribution is

$$p_{\theta}(\cdot \mid s) = \text{softmax}(\ell)$$

The value means:

$$v_{\theta}(s) \approx \begin{cases} +1 & \text{current player wins} \\ 0 & \text{draw} \\ -1 & \text{current player loses} \end{cases}$$

MCTS as a Function

MCTS takes a network and a root state:

$$\text{MCTS}_{\theta}(s_0) \rightarrow \pi$$

where

$$s_0 = (B_0, c_0)$$

and

$$\pi(a \mid s_0)$$

is the search-improved policy.

The final MCTS policy is computed from root visit counts:

$$\pi(a \mid s_0) = \frac{N_{s_0}(a)}{\sum_b N_{s_0}(b)}$$

Node Statistics

For each node s and legal action a , MCTS stores:

$P_s(a)$ = prior probability

$N_s(a)$ = visit count

$W_s(a)$ = total backed-up value

$$Q_s(a) = \frac{W_s(a)}{\max(1, N_s(a))}$$

These are edge statistics:

(s, a)

not just node statistics.

When MCTS reaches an unexpanded node $s = (B, c)$, it calls the network:

$$p, v = f_{\theta}(\text{encode}(B, c))$$

Mask illegal actions:

$$p_{\text{legal}}(a) = \begin{cases} p(a) & a \in A(s) \\ 0 & a \notin A(s) \end{cases}$$

Renormalize:

$$P_s(a) = \frac{p_{\text{legal}}(a)}{\sum_{b \in A(s)} p_{\text{legal}}(b)}$$

So the policy head becomes the MCTS prior.

Leaf Evaluation

The value head evaluates the leaf:

$$v_{\theta}(s)$$

In this implementation, values are stored from the root player's perspective.

Let root player be c_0 . Then:

$$V_{\text{root}}(s) = \begin{cases} v_{\theta}(s) & c = c_0 \\ -v_{\theta}(s) & c \neq c_0 \end{cases}$$

Thus MCTS uses the value network instead of random rollouts:

$$\text{leaf} \rightarrow v_{\theta}(\text{leaf})$$

rather than

leaf $\rightarrow$ random game to terminal.

Selection: PUCT

At an expanded node s , MCTS selects:

$$a^* = \arg \max_{a \in A(s)} \left[Q_s^{\text{select}}(a) + U_s(a) \right]$$

where

$$U_s(a) = c_{\text{puct}} P_s(a) \frac{\sqrt{\sum_b N_s(b) + 1}}{1 + N_s(a)}$$

Interpretation:

$Q_s(a)$: exploit good actions

$U_s(a)$: explore promising or under-visited actions

UCT vs. PUCT

Traditional MCTS often uses UCT:

$$\text{score}_{\text{UCT}}(a) = Q_s(a) + c \sqrt{\frac{\log \sum_b N_s(b)}{N_s(a)}}$$

Exploration is driven mainly by visit counts.

AlphaZero-style MCTS uses PUCT:

$$\text{score}_{\text{PUCT}}(a) = Q_s(a) + c_{\text{puct}} P_s(a) \frac{\sqrt{\sum_b N_s(b) + 1}}{1 + N_s(a)}$$

Here,

$P_s(a)$ = policy prior from the neural network.

PUCT explores under-visited moves that the network considers promising.

Perspective Correction

The code stores $Q_s(a)$ from the root player's perspective.
But at node $s = (B, c)$, the current player chooses the action.
Therefore:

$$Q_s^{\text{select}}(a) = \begin{cases} Q_s(a) & c = c_0 \\ -Q_s(a) & c \neq c_0 \end{cases}$$

This makes each player select actions that are good for themselves.

One MCTS Simulation

One simulation:

$$s \leftarrow s_0$$

$$\text{Path} \leftarrow []$$

Repeat:

if s terminal: $V \leftarrow \text{TerminalValue}(s, c_0)$

else if s unexpanded: $V \leftarrow \text{Expand}_\theta(s, c_0)$

else: $a \leftarrow \text{Select}(s)$, $\text{Path} \leftarrow \text{Path} \cup (s, a)$, $s \leftarrow T(s, a)$

Then backpropagate V along the path.

Backpropagation

For every edge (s_t, a_t) in the selected path:

$$N_{s_t}(a_t) \leftarrow N_{s_t}(a_t) + 1$$

$$W_{s_t}(a_t) \leftarrow W_{s_t}(a_t) + V$$

$$Q_{s_t}(a_t) = \frac{W_{s_t}(a_t)}{N_{s_t}(a_t)}$$

The same simulation value V is backed up through the selected path.

Full MCTS Procedure

Given root state s_0 :

$$\text{Expand}_{\theta}(s_0)$$

Run S simulations:

$$\text{for } j = 1, \dots, S : \quad \text{Simulate}_{\theta}(s_0)$$

Return normalized root visit counts:

$$\pi(a \mid s_0) = \frac{N_{s_0}(a)}{\sum_b N_{s_0}(b)}$$

This π is the MCTS-improved policy.

Self-Play

At each real game state s_t :

$$\pi_t = \text{MCTS}_{\theta}(s_t)$$

Choose action:

$$a_t \sim \pi_t$$

Store:

$$(s_t, \pi_t)$$

Then apply the move:

$$s_{t+1} = T(s_t, a_t)$$

The self-play game continues until terminal.

Replay Buffer

Self-play generates one trajectory:

$$(s_0, \pi_0), (s_1, \pi_1), \dots, (s_T, \pi_T)$$

After the game ends, each state receives an outcome target:

$$z_t \in \{-1, 0, +1\}$$

The replay buffer stores training examples:

$$R = \{(x_i, \pi_i, z_i)\}$$

where

$$x_i = \text{encode}(B_i, c_i)$$

The buffer keeps examples from many recent games.

Purpose:

train on a diverse mixture of past positions

rather than only the latest game.

Training Targets

After the game ends, let winner be w .

For each stored state

$$s_t = (B_t, c_t)$$

define

$$x_t = \text{encode}(B_t, c_t)$$

and value target:

$$z_t = \begin{cases} +1 & w = c_t \\ 0 & w = 0 \\ -1 & w = -c_t \end{cases}$$

Each replay example is:

$$(x_t, \pi_t, z_t)$$

Training Loss

Network prediction:

$$p_t, v_t = f_{\theta}(x_t)$$

Policy loss:

$$\mathcal{L}_{\text{policy}} = - \sum_a \pi_t(a) \log p_t(a)$$

This is equivalent to minimizing:

$$\text{KL}(\pi_t \| p_t)$$

Value loss:

$$\mathcal{L}_{\text{value}} = (v_t - z_t)^2$$

Total loss:

$$\mathcal{L} = \mathcal{L}_{\text{policy}} + \mathcal{L}_{\text{value}}$$

Training Loop

$\theta \leftarrow$ initialize network

$R \leftarrow \emptyset$

For each episode:

play one full self-play game using MCTS_{θ}

$R \leftarrow R \cup \{(x_t, \pi_t, z_t)\}$

Sample minibatches from R , then update:

$\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}$

So:

MCTS improves targets, training distills them into f_{θ} .

Core Feedback Loop

$f_{\theta}(s) \rightarrow$ raw policy and value

$\text{MCTS}_{\theta}(s) \rightarrow$ search-improved policy π

self-play $\rightarrow (x, \pi, z)$

training $\rightarrow$ better f_{θ}

Main idea:

The network guides search; search trains the network.

Thanks for your attention!